Clautolisp Implementation Roadmap

Codex

Purpose

This document is the architectural plan for bringing `clautolisp` from its current Phase-6 state (reader + evaluator + small core builtins, dialect-aware) to broad coverage of the AutoLISP / Visual LISP language family documented in `autolisp-spec`.

It distinguishes:

- **function coverage** — adding pure-language builtins that do not touch a CAD host (numeric, list, string, control, error);
- **host coverage** — adding the host-facing surface that real AutoLISP code depends on (entities, selection sets, tables, sysvars, command dispatch, graphics, dialogs, reactors);
- **backends** — the runtime swappable implementations of the host surface: a deterministic **MockHost**, an optional **GuiHost**, and a future **LiveHost** adapter to BricsCAD or AutoCAD.

The plan is intentionally headless-first: every layer must be testable without a graphical CAD process. A GUI backend is a later opt-in, not a prerequisite.

The plan is intentionally driven by the spec, not by an internal wishlist: each phase below references the chapters of the `autolisp-spec` document that define its scope.

Strategic Position

The architectural choice from chapter 1 of the spec stands:

1. **Implement AutoLISP as a Common-Lisp evaluator, not as a translation into Common Lisp.**
2. **Common Lisp is the implementation and extension language.** It does not provide the semantics.
3. *Host effects go through a single abstract interface (the HAL — Host Abstraction Layer) so that backends can be swapped at session-creation time.*

The dialect descriptor introduced in Phase 6 (`strict`, `autocad-2026`, `bricscad-v26`) controls **language-level** product divergence. The HAL choice is **orthogonal**: a dialect describes what the source code is allowed to mean; a backend describes where its host effects land.

Layered Architecture

```
+----------------------------------------+
|  AutoLISP source (your application)    |
+----------------------------------------+
|         Reader  (autolisp-reader)      |   chapter 2
+----------------------------------------+
|        Evaluator (autolisp-runtime)    |   chapter 3
+----------------------------------------+
|     Builtin registry (-builtins-core) |   chapters 4-7, 9-15
+----------------------------------------+
|  HAL - Host Abstraction Layer (NEW)    |   chapters 16, 18-22
+----------------------------------------+
|   Backend: MockHost | GuiHost (opt)    |
|            LiveHost (deferred)         |
+----------------------------------------+
```

The reader, runtime, and core builtins exist today and are sufficient for any program whose surface is values + control flow. The HAL is the next architectural addition; everything beyond it is backend-specific work.

Data Model — Inferred From the CAD Specifications

Most AutoLISP host functions operate on a small set of recurring data shapes. The runtime needs explicit Common-Lisp structures for each, even when the active backend stores nothing real behind them.

Entity (ENAME)

A handle naming a single CAD object. AutoLISP code obtains an ENAME from `entlast`, `entnext`, `ssname`, etc., and treats it opaquely. It is round-tripped through `entget` (returns a **DXF group-code list** — a list of dotted pairs of group-code integers and values) and `entmod` / `entupd` / `entdel` / `entmake` / `entmakex`.

```
(defstruct entity-handle
  (id      (gensym "ENT-") :type t)   ; opaque key into the backend
  (kind    nil :type symbol)          ; :line / :circle / :insert / ...
  (block   nil)                       ; owning block-table-record handle
```

```
  (layer   nil)                        ; layer-table-record handle
  (data    nil :type list)             ; cached DXF group-code list
  (deleted nil :type boolean))
```

The DXF group-code list (`data`) is the primary AutoLISP-visible representation: `(0 . "LINE") (5 . "1A") (8 . "0") (10 5.0 5.0 0.0) (11 7.0 5.0 0.0)` etc. Every entity-modifying builtin works against that shape; the backend is free to keep additional fields (graphic state, ObjectId, etc.) on the side.

Selection set (PICKSET)

A bag of ENAMEs with insertion-order-preserving identity. Built by `ssget` from filter lists; manipulated by `ssadd`, `ssdel`, `sslength`, `ssname`. `ssgetfirst` / `sssetfirst` are pickfirst hooks.

```
(defstruct pickset
  (id      (gensym "SS-") :type t)
  (members nil :type list))           ; ordered list of entity-handle
```

Tables (block, layer, ltype, style, view, ucs, vport, dimstyle, appid)

Symbol tables in DXF terminology. `tblnext`, `tblsearch`, `tblobjname`. `entget` on a table-record returns the same DXF list shape.

```
(defstruct symbol-table-record
  (id      (gensym "TBL-") :type t)
  (kind    nil :type symbol)          ; :block / :layer / :ltype / ...
  (name    "" :type string)
  (data    nil :type list))
```

The session keeps one **symbol-table** per kind, indexed by name.

Dictionaries

Generic key→object containers anchored at the named-object dictionary. `namedobjdict`, `dictsearch`, `dictnext`, `dictadd`, `dictremove`, `dictrename`. Dictionary entries can be entities, xrecords, or sub-dictionaries.

```
(defstruct dictionary
  (id      (gensym "DICT-") :type t)
  (entries (make-hash-table :test #'equal)))   ; string -> handle
```

Symbol tables for system variables

`getvar` / `setvar` access named state. `initget` / `getstring` / `getpoint` etc. read interactive input. The session needs a sysvar table:

```
(defstruct sysvar-table
  (entries (make-hash-table :test #'equalp)))   ; name -> sysvar-cell

(defstruct sysvar-cell
  (name    "" :type string)
  (kind    nil :type symbol)          ; :integer / :real / :point / :string / :short
  (value   nil)
  (read-only-p nil :type boolean))
```

SAFEARRAY / VARIANT

The Visual LISP COM bridge. SAFEARRAYs are typed multi-dimensional arrays; VARIANTs tag a value with its COM type. Modeled now (the runtime already has `autolisp-safearray`, `autolisp-variant`, `autolisp-vla-object` stubs); operators in the `vlax-*` family will fill them in.

VLA-object

A handle to a COM-exposed object (Layer, Block, Document, etc.). The runtime stores an opaque ID into a per-session table; the backend decides what the ID means.

```
(defstruct vla-object-handle
  (id          (gensym "VLA-") :type t)
  (kind        nil :type symbol)      ; :document / :layer / :block / :line / ...
  (entity-back nil)                   ; ename if applicable
  (released-p  nil :type boolean))
```

Layer states

Persistent named snapshots of layer-property bundles. `layerstate-*` family.

Reactors

Object-, document-, and editor-level callback registrations. Modeled per `vlr-*` entry: a callback list keyed by event class.

DCL dialog

The Dialog Control Language exposes a tile tree built from a .dcl file. Out of scope for the headless-first phase; tracked here so the data shape is visible (tile, action, list, image controls).

The Host Abstraction Layer (HAL)

Goal

Decouple the **what** (what the AutoLISP builtin does) from the **where** (which CAD backend stores or executes its effect).

Shape

A small set of generic functions per data domain. Each backend provides a method on every operation it supports; unsupported operations signal a uniform `:host-not-supported` runtime error.

```
(defgeneric host-entget        (host ename))
(defgeneric host-entmod        (host ename group-code-list))
(defgeneric host-entmake       (host group-code-list))
(defgeneric host-entdel        (host ename))
(defgeneric host-entlast       (host))
(defgeneric host-entnext       (host ename))

(defgeneric host-ssget         (host filter &key mode))
(defgeneric host-ssadd         (host pickset ename))
(defgeneric host-ssname        (host pickset index))
(defgeneric host-sslength      (host pickset))

(defgeneric host-tblsearch     (host table-kind name))
(defgeneric host-tblnext       (host table-kind &key rewind))

(defgeneric host-getvar        (host name))
(defgeneric host-setvar        (host name value))

(defgeneric host-command       (host argv))   ; the (command ...) builtin
(defgeneric host-prompt        (host string))
(defgeneric host-getstring     (host prompt &key controls))
(defgeneric host-getpoint      (host prompt &key base controls))
(defgeneric host-getreal       (host prompt &key controls))
(defgeneric host-getint        (host prompt &key controls))

(defgeneric host-grdraw        (host from to colour highlight))
(defgeneric host-grtext        (host kind ...))
(defgeneric host-grclear       (host))
```

Per-session backend

The `runtime-session` struct (autolisp-runtime/source/model.lisp) gains a slot:

```
(defstruct runtime-session
```

```
...
(host nil))
```

A new constructor argument `:host` defaults to a **NullHost** singleton that errors on every operation; production callers pass a MockHost or, eventually, a LiveHost.

Error codes

Existing runtime error codes (`:invalid-list-argument`, etc.) extend with:

- `:host-not-supported` — backend does not implement this operation
- `:host-not-connected` — operation requires a live CAD process
- `:host-protocol-error` — backend returned a malformed reply

Backends

NullHost

Trivial. Every generic function method signals `:host-not-supported`. Default for the strict and other test sessions; preserves current behaviour.

MockHost

In-memory deterministic CAD-database substitute. Owns:

- one `entity-table` (hash-table from ename id → entity-handle)
- one `pickset-table`
- one `vla-object-table`
- per-kind ~symbol-table~s for blocks, layers, ltypes, styles, views, ucs, vports, dimstyles, appids
- one `named-object-dictionary`
- one `sysvar-table` pre-populated with conservative defaults
- one `prompt-stream` (input source for `getstring` etc.)
- one `display-log` (records `grdraw` / `grtext` calls for inspection from tests)

The MockHost is the production runtime substrate for every clautolisp scenario that does not have a live CAD process behind it. It must be deterministic: the same input script under the same dialect, the same MockHost initial state, and the same `prompt-stream` must produce the same output and the same final database state.

GuiHost (optional, future)

A standalone Qt or web view that talks to clautolisp through the HAL via a JSON / S-expression protocol. Headless-test-friendly: the GUI is an observer of MockHost state, not a substitute for it.

LiveHost

A bridge to BricsCAD or AutoCAD. Speculative for now. Two plausible shapes:

- **Stdio bridge** — clautolisp drives an interactive AutoLISP REPL inside a real CAD process and reads back tagged S-expressions. Closest to the existing `autolisp-script` harness used for spec probes.
- **COM bridge** — Windows-only; the runtime drives the CAD process through its OLE Automation interface. More direct but Windows-bound.

Either way, LiveHost reuses the HAL contract: existing AutoLISP source runs unchanged when the backend is swapped.

Phased Roadmap

Numbering continues from Phase 6 (the Lisp-1, dynamic-scope, and general-semantics work that closed in 2026-04-26).

Phase 7 — Function-Coverage Round-Out

Goal: implement every host-independent builtin so that pure-data AutoLISP source loads cleanly. No HAL changes; the existing NullHost remains the default.

Scope (drives by `autolisp-spec` chapters 4-7, 9-13):

- Numeric / math: =sqrt, exp, log, log10, sin, cos, tan, asin, acos, atan, atan2 form, expt, mod, abs (already), random, floor, ceiling, round, fix (already)=
- Bitwise: `boole, ~ (already), logand (already), logior (already), logxor, lsh (already)`
- List: =append (already), reverse (already), nth (already), member (already), assoc (already), last (already), length (already), list (already), cons (already), subst (already), vl-list-length, vl-position, vl-remove (already), vl-sort, vl-sort-i, distance, remove= and the smaller corners.
- String: family already substantially in place; round out `vl-string-elt`, `vl-string-translate`, `vl-string-position` (done), `snvalid`, `wcmatch`, `xstrcase`.
- Predicate: `atom` (done), `numberp` (done), `zerop` (done), `minusp` (done), `listp` (done), `vl-consp` (done), `vl-symbolp` (done), `atoms-family`.
- Symbol: `boundp` (done), `vl-symbol-name` (done), `vl-symbol-value` (done), `set` (done), `atoms-family` (todo).
- I/O: `princ`, `prin1`, `print`, `terpri`, `read-line`, `read-char`, `write-line`, `write-char`, `open`, `close` (all done).
- Error / control: `vl-catch-all-apply` (done), `exit` / `quit` (done), `*error*` hook (done), `setfunhelp`, `vl-bt`, `vl-bt-on`, `vl-bt-off`.

Deliverables:

- one or two new sections in `autolisp-builtins-core/source/api.lisp`

- one new test file per family in `autolisp-builtins-core/tests/`
- coverage table (autogenerated from `core-builtins` vs the spec entries) in this document

Phase 8 — HAL Foundation [DONE 2026-04-26]

Goal: introduce the abstract host interface and the NullHost default. No behaviour change; a structural step.

Status: closed in commit **{phase-8}**. Suite 818 / 0; smoke-test of the existing real-world workflow runs unchanged.

Scope delivered:

- New ASDF system `clautolisp/autolisp-host` depending on `clautolisp/autolisp-runtime`. Three source files: `package.lisp`, `protocol.lisp` (every generic-function signature), `null-host.lisp` (the NullHost class + every method).
- Generic-function surface covers entity (`entget`, `entmod`, `entmake`, `entdel`, `entlast`, `entnext`, `handent`), selection set (`ssget`, `ssadd`, `ssdel`, `ssname`, `sslength`, `ssmemb`, `ssgetfirst`, `sssetfirst`), tables (`tblsearch`, `tblnext`, `tblobjname`), dictionaries (`namedobjdict`, `dictsearch`, `dictnext`, `dictadd`, `dictremove`, `dictrename`), sysvars (`getvar`, `setvar`), command dispatch (`command`), prompts (`prompt`, `initget`, `getstring`, `getint`, `getreal`, `getpoint`, `getcorner`, `getdist`, `getangle`, `getorient`, `getkword`), and graphics (`grdraw`, `grtext`, `grvecs`, `grclear`, `grread`, `redraw`).
- Runtime: `runtime-session` gains a `host` slot; new accessors `runtime-session-host`, `set-runtime-session-host`, `current-evaluation-host`. The `*default-runtime-host*` parameter holds the process-wide fallback; the autolisp-host module installs the NullHost singleton there at load time.
- New runtime error code: `:host-not-supported` (signalled by every NullHost method, with the backend name and operation name embedded in the message).
- CLI: new `--host NAME` flag, accepted values `null` (default), with `mock` reserved for Phase 9 (errors with a clear message). The REPL banner now reads "`clautolisp X.Y.Z - REPL (DIALECT dialect, HOST host)`". Builtins installation runs after the host is attached so future host-aware builtins see the right backend.
- Tests: 14 new tests in a fresh `clautolisp/autolisp-host/tests` system pin every NullHost method's error code, the default installation, the per-session keyword override, and `set-runtime-session-host`. Aggregate suite 818 / 0.

Note: no existing core builtin currently calls out to the host layer, so the "re-route" step listed in the original Phase 8 scope was a no-op. Phase 9 will introduce the first builtins that actually use the HAL.

Phase 9 — MockHost Data Structures [DONE 2026-04-26]

Goal: the in-memory substrate for the entity / pickset / table / sysvar layers. No host operations yet; only the data carriers.

Status: closed. Suite 883 / 0; `clautolisp --host mock` now constructs a usable MockHost (host operations still error with `:host-not-supported` until Phase 10 fills in the methods).

Scope delivered:

- New ASDF system `clautolisp/autolisp-mock-host` depending on `clautolisp/autolisp-host` and `clautolisp/autolisp-runtime`. Four source files: `package.lisp`, `model.lisp` (struct defs + the `mock-host` class), `sysvars.lisp` (default tables and sysvars), `api.lisp` (constructor + snapshot/restore).
- Structs delivered as sketched in the data-model section: `entity-handle`, `pickset`, `symbol-table-record`, `dictionary`, `sysvar-cell`, plus the `mock-host` class itself with slots for entities, picksets, vla-objects, per-kind table hash, named-object dictionary, sysvars, prompt-stream, prompt-output, display-log, pickfirst.
- Pre-populated symbol tables: `:block-record` holds `"*Model_Space"` / `"*Paper_Space"`; `:layer` holds `"0"`; `:ltype` holds `"BYBLOCK"` / `"BYLAYER"` / `"Continuous"`; `:style` and `:dimstyle` each hold `"Standard"`; `:vport` holds `"*Active"`; `:appid` holds `"ACAD"`. `:ucs` and `:view` start empty.
- Pre-populated sysvars (CMDECHO, CECOLOR, CLAYER, ANGBASE, ANGDIR, AUNITS, OSMODE; USERR1..R5, USERI1..I5, USERS1..S5; plus read-only DWGNAME / DWGPREFIX / PLATFORM and writable LISPSYS). Lookup is case-insensitive; writes to read-only cells signal `:host-not-supported`.
- `mock-host-snapshot` / `mock-host-restore` pair: the snapshot is a property list of entities, picksets, tables, sysvars, and pickfirst pointer; round-trip restore preserves the host's AutoLISP-visible state. Tests pin determinism.
- Convenience accessors: `mock-host-table`, `mock-host-find-table-record`, `mock-host-add-table-record`, `mock-host-sysvar`, `mock-host-set-sysvar`.
- Base-class fallback methods on `host` for every HAL operation signalling `:host-not-supported`. This lets MockHost (with no method overrides yet) inherit a sensible "not yet implemented" failure shape; Phase 10 will override the methods that MockHost actually serves.
- CLI: `--host mock` now constructs a fresh MockHost (instead of erroring as a Phase-9-todo). The REPL banner reads "...REPL (DIALECT dialect, mock-host host)~~~". Host operations still error until Phase 10.
- Tests: 14 new tests in `clautolisp/autolisp-mock-host/tests` cover class hierarchy, empty fixture, default tables, default sysvars, sysvar mutation including read-only refusal, case-insensitive lookup, snapshot/restore round-trip, slot shapes for prompt-output / display-log / pickset / dictio-

nary, and the inherited `:host-not-supported` behaviour. Aggregate suite 76 + 184 + 29 + 52 + 412 + 130 = 883 / 0.

Phase 10 — MockHost Entity API [DONE 2026-04-26]

Goal: implement the entity-level HAL methods against MockHost.

Status: closed. Suite 919 / 0; the AutoLISP-visible entity round trip (entmake -> entget / entlast / entnext / handent / entmod / entdel) runs end-to-end against `--host mock`.

Scope delivered:

- New file `clautolisp/autolisp-mock-host/source/entity-api.lisp` defines methods on `mock-host` for `host-entget`, `host-entmod`, `host-entmake`, `host-entmakex`, `host-entdel` (with the documented within-command toggle), `host-entupd`, `host-entlast`, `host-entnext` (both zero-arg and one-arg forms), `host-handent`.
- `mock-host` class extended with a hex-string handle allocator (`mock-host-allocate-handle`), a `creation-order` list (walked by entlast / entnext), and an `entities` hash-table re-keyed on the handle string.
- `entmake` / `entmakex` accept a DXF group-code list with a `(0 . "TYPE")` marker; the host injects `(-1 . ENAME)` at the head and `(5 . HANDLE)` as the second entry, mirroring AutoCAD's on-disk shape, and stores the resulting list. Missing marker signals `:invalid-entity-data`.
- `entget` on a deleted entity returns nil; `entdel` toggles the `deleted-p` flag (re-entering "live" on a second call within the same scope, as documented).
- `handent` accepts both CL strings and AutoLISP-strings, returns an ENAME for known handles and nil otherwise.
- `clautolisp/autolisp-builtins-core` now depends on `clautolisp/autolisp-host`. New AutoLISP builtins `ENTGET`, `ENTMOD`, `ENTMAKE`, `ENTMAKEX`, `ENTDEL`, `ENTUPD`, `ENTLAST`, `ENTNEXT`, `HANDENT` are thin wrappers over the HAL generic functions, dispatched on the active session's host backend.
- `autolisp-runtime` now exports `make-autolisp-ename` (the public ENAME constructor used by the HAL).
- The `autolisp-value->string` printer renders ENAMEs as "<Entity name: HANDLE>" so the REPL banner displays them cleanly under `--host mock`.
- snvalid at the host level (cross-checking against current table-record names) is parked for Phase 11 alongside tblsearch / tblnext, since it relies on table walkers.

Tests:

- 11 new entity-api tests in `autolisp-mock-host/tests/entity-api-tests.lisp` pin every method's contract: handle allocation, bookkeeping injection, round-trip via entget, delete-toggle, entmod-replace, creation-order walk

under entlast / entnext, deleted-skip, handent lookup, invalid-data error path, invalid-ename error path.
- 1 new builtin-level test in autolisp-builtins-core/tests pins the AutoLISP-side wiring under both NullHost and MockHost.
- Aggregate suite 76 + 184 + 29 + 82 + 418 + 130 = 919 / 0.

Smoke: `clautolisp --host mock -x "(setq e (entmake ...)) (entget (cdr (assoc -1 e)))"` runs the entmake -> entget round trip end-to-end and prints the layer name from the stored data list.

Phase 11 — MockHost Selection / Tables / Sysvars [DONE 2026-04-26]

Goal: selection sets, table walkers, and system variable access on MockHost.

Status: closed. Suite 959 / 0; the AutoLISP-side ‘(ssget "X" filter)‘ / ‘(tblsearch . . .)‘ / ‘(getvar . . .)‘ / ‘(setvar . . .)‘ flow runs end-to-end against `--host mock`.

Scope delivered:

- New file `autolisp-mock-host/source/selection-api.lisp` implements `host-ssget` (modes "X" / "_X" with optional filter-list; other modes signal :unsupported-ssget-mode), `host-ssadd` (zero / one / two arg shapes), `host-ssdel`, `host-ssname` (zero-based), `host-sslength`, `host-ssmemb`, `host-ssgetfirst`, `host-sssetfirst`. Filter-list matching honours comma-separated alternation and the ‘*‘ / ‘¿ wildcards on string values. Group-code -4 logical operators (<AND, <OR, <NOT) are deferred.
- New file `autolisp-mock-host/source/table-api.lisp` implements `host-tblsearch`, `host-tblnext` (with per-kind iterator state on `mock-host-tblnext-iterators`; rewind restarts), and `host-tblobjname` returning an ENAME wrapping the table-record's internal id.
- New file `autolisp-mock-host/source/sysvar-api.lisp` implements `host-getvar` (wraps strings in autolisp-string for AutoLISP consumption) and `host-setvar` (per-kind coercion: integers truncate reals; reals coerce numbers; strings reject non-strings; points validate 2D/3D coordinate lists). Read-only cells signal :sysvar-read-only; unknown names signal :unknown-sysvar.
- `autolisp-builtins-core` gained 13 new AutoLISP builtins routing through the HAL: `SSGET`, `SSADD`, `SSDEL`, `SSNAME`, `SSLENGTH`, `SSMEMB`, `SSGETFIRST`, `SSSETFIRST`, `TBLSEARCH`, `TBLNEXT`, `TBLOBJNAME`, `GETVAR`, `SETVAR`.
- Runtime exports new constructor `make-autolisp-pickset`.

Tests:

- 21 new tests in `autolisp-mock-host/tests/selection-tests.lisp` pin: ssget X / X+filter / type / layer / comma-alternation / wildcard / empty result / no-mode / unsupported-mode; ssadd / ssdel / ssname positional / ssmemb membership / ssgetfirst+sssetfirst round-trip; tblsearch hit + miss;

tblnext walk and rewind; tblobjname returns ename; getvar with string wrapping; getvar on unknown name returns nil; setvar coerces integer / real / string; setvar rejects read-only / unknown / type-mismatch.
- Aggregate suite 76 + 184 + 29 + 122 + 418 + 130 = 959 / 0.

Smoke (from the CLI under `--host mock`):

```
clautolisp --host mock -x \
  "(princ (getvar \"CLAYER\"))            ; -> 0
   (setvar \"CLAYER\" \"Drafting\")
   (princ (getvar \"CLAYER\"))            ; -> Drafting
   (princ (cdr (assoc 2 (tblsearch \"LAYER\" \"0\"))))  ; -> 0
   (entmake (list (cons 0 \"LINE\") (cons 8 \"0\")))
   (entmake (list (cons 0 \"CIRCLE\") (cons 8 \"0\")))
   (princ (sslength (ssget \"X\" (list (cons 0 \"LINE\")))))"  ; -> 1
```

Phase 12 — Headless Interaction Channel [DONE 2026-04-26]

Goal: let real AutoLISP scripts that read user input run in batch.

Status: closed. Suite 979 / 0; the AutoLISP-side `(getstring)` / `(getreal)` / `(getpoint)` / `(initget BITS KW)` `(getkword)` flow runs end-to-end against `--host mock`, with deterministic answers fed via `--mock-input PATH`. The CLI's default `--host` is now `mock` (was `null`), so the common `(load FILE.lsp)` workflow no longer fails with `:host-not-supported` on a bare `(getvar ...)` call.

Scope delivered:

- New file `autolisp-mock-host/source/prompt-api.lisp` implements `host-prompt` (writes to `mock-host-prompt-output`), `host-initget` (records bits + keyword list on `mock-host-pending-initget` as a one-shot value consumed by the next get*), `host-getstring`, `host-getint`, `host-getreal`, `host-getpoint` (parses 2-D / 3-D coordinate lists; pads Z=0 for 2-D), `host-getcorner`, `host-getdist`, `host-getangle`, `host-getorient`, `host-getkword` (matches against the initget keyword list when present, otherwise returns the trimmed line). Each get* reads one line from `mock-host-prompt-stream`; EOF / unset stream returns nil.
- `mock-host` class extended with a `pending-initget` slot and the `initget-state` helper class.
- 11 new AutoLISP builtins: `INITGET`, `GETSTRING`, `GETINT`, `GETREAL`, `GETPOINT`, `GETCORNER`, `GETDIST`, `GETANGLE`, `GETORIENT`, `GETKWORD`. PROMPT was already a builtin (writes to standard output) and is left untouched in this phase.
- CLI: `--mock-input PATH` flag attaches the file's lines as the MockHost's prompt-stream so headless test rigs can feed deterministic answers. The

`--host` default flips from `null` to `mock`; specifying `--host null` explicitly preserves the strict no-host error path for tests / static analysis.
- `grread` remains unimplemented; it is parked alongside the graphics layer for Phase 13+.

Tests:

- 14 new tests in `autolisp-mock-host/tests/prompt-tests.lisp` pin: prompt output writes; getstring reads one line; EOF returns nil; getint parses integer / rejects non-integer; getreal parses double-float; getpoint parses 2D (pads Z) and 3D; getdist / getangle parse reals; getkword matches the initget list / rejects non-matches / returns line when no initget; initget is one-shot (consumed by next get*); prompt text precedes the input.
- Aggregate suite 76 + 184 + 29 + 142 + 418 + 130 = 979 / 0.

Smoke (under the new default `--host mock`):

```
$ clautolisp -x "(princ (getvar \"CLAYER\"))"           ; -> 0
$ echo "Yes 42 1.5 2.5" > /tmp/in.txt
$ clautolisp --mock-input /tmp/in.txt \
             -x "(princ (getstring))"                  ; -> Yes 42 1.5 2.5
```

Phase 13 — VLA / VLAX Foundations [DONE 2026-04-26]

Goal: the COM compatibility surface, headless-first.

Status: closed. Suite 1008 / 0; the AutoLISP-side `vlax-create-object` / `vlax-get-property` / `vlax-put-property` / `vlax-invoke-method` / `vlax-release-object` flow round-trips against `--host mock`. SAFEARRAY and VARIANT round trip through their own builtins entirely in pure CL.

Why headless-only: a real Windows-COM bridge is not possible on macOS / Linux without Wine. .NET being installed does not unlock COM on non-Windows hosts; the Microsoft COM marshaller raises `PlatformNotSupportedException` outside Windows. No mainstream library implements the Windows COM API for native macOS / Linux processes (Samba's MS-RPC stack, Mono's interop shims, XPCOM, GObject, D-Bus, CORBA — none speak AutoCAD's COM type-library ABI). Real-COM bridging is therefore deferred to Phase 16 (LiveHost), where it is gated behind the user actually being on Windows with AutoCAD installed (or driving BricsCAD on macOS via the existing autolisp-script stdio bridge).

Scope delivered:

- `clautolisp/autolisp-host` gains 8 new HAL generic functions: `host-vlax-create-object`, `host-vlax-get-object`, `host-vlax-release-object`, `host-vlax-get-property`, `host-vlax-put-property`, `host-vlax-invoke-method`, `host-vlax-property-available-p`, `host-vlax-method-applicable-p`. Each has a base-class fallback signalling :host-not-supported.

- New file `autolisp-mock-host/source/com-progids.lisp` registers a small declarative ProgID table seeded with `"AutoCAD.Application"`, `"AutoCAD.Document"`, and `"AutoCAD.Layers"`. Each template lists default property values and method handlers. `register-com-progid` extends the registry from user code or tests.
- New file `autolisp-mock-host/source/vlax-api.lisp` implements every host-vlax-* method on MockHost. AutoLISP-visible VLA-objects wrap a host-allocated COM-object id; the host stores the mock-com-object in a hash-table keyed on that id. Released objects raise :released-vla-object on subsequent property/method access; unknown ProgIDs raise :unknown-progid; unknown property/method names raise :unknown-com-property / :unknown-com-method.
- 22 new AutoLISP builtins in `clautolisp/autolisp-builtins-core`:
  - COM-object: `VLAX-CREATE-OBJECT`, `VLAX-GET-OBJECT`, `VLAX-GET-OR-CREATE-OBJECT`, `VLAX-RELEASE-OBJECT`, `VLAX-OBJECT-RELEASED-P`, `VLAX-GET-PROPERTY`, `VLAX-PUT-PROPERTY`, `VLAX-INVOKE-METHOD`, `VLAX-PROPERTY-AVAILABLE-P`, `VLAX-METHOD-APPLICABLE-P`.
  - SAFEARRAY: `VLAX-MAKE-SAFEARRAY`, `VLAX-SAFEARRAY-FILL`, `VLAX-SAFEARRAY-PUT-ELEMENT`, `VLAX-SAFEARRAY-GET-ELEMENT`, `VLAX-SAFEARRAY->LIST`, `VLAX-SAFEARRAY-TYPE`, `VLAX-SAFEARRAY-GET-L-BOUND`, `VLAX-SAFEARRAY-GET-U-BOUND`.
  - VARIANT: `VLAX-MAKE-VARIANT`, `VLAX-VARIANT-TYPE`, `VLAX-VARIANT-VALUE`, `VLAX-VARIANT-CHANGE-TYPE`.

  SAFEARRAY storage is an internal `safearray-data` struct (type-tag + bound pairs + 1-D CL backing array; flat indexing handles arbitrary dimensions). VARIANT storage is a tagged cons (type-keyword . value).
- Runtime exports `make-autolisp-vla-object`, `make-autolisp-safearray`, `make-autolisp-variant` from the public API.
- Printer: VLA-objects render as `#<VLA-OBJECT id>`; SAFEARRAY shows its type-tag and bounds; VARIANT shows its type-tag and inner value.

Out of scope (deferred):

- `vlax-import-type-library` — left to a later sub-phase; needs a parsed type-library ingestion pipeline.
- `vla-*` wrapper generation — large surface; defer until a declarative ProgID-method table emerges from real workloads.
- Real Windows COM bridge — Phase 16 (LiveHost), Windows-only.

Tests:

- 11 new tests in `autolisp-mock-host/tests/vlax-tests.lisp` pin every VLAX HAL method's contract, including the released-object guard, unknown-ProgID, unknown-property, unknown-method, and the dynamic registry extension via register-com-progid.
- 1 new builtin-level test pins the AutoLISP-side wiring for vlax-create-object / get-property / put-property / invoke / release / object-released-p plus SAFEARRAY and VARIANT round-trips.

- Aggregate suite 76 + 184 + 29 + 161 + 428 + 130 = 1008 / 0.

Smoke (under the default `--host mock`):

```
$ clautolisp -x \
  "(setq app (vlax-create-object \"AutoCAD.Application\"))
   (princ (vlax-get-property app \"Name\"))            ; -> Mock AutoCAD
   (princ (vlax-property-available-p app \"Visible\")) ; -> T
   (princ (vlax-method-applicable-p app \"Quit\"))     ; -> T
   (vlax-release-object app)
   (princ (vlax-object-released-p app))"               ; -> T
```

Phase 14 — Reactors [DONE 2026-04-26]

Goal: the Visual LISP reactor surface, on top of an explicit host-object ontology and lifecycle vocabulary.

Status: closed. Suite 1037 / 0. Reactors are dispatched by the runtime against per-document and per-application registries; MockHost mutations emit canonical events into the channel; the full `vlr-*` AutoLISP-visible family is implemented; a complete end-to-end test wires entmake / entmod / setvar through to registered reactors.

Scope delivered, split across two sub-phases:

Phase 14a — Ontology + event channel

- `clautolisp/documentation/design.org` — design document defining the host-object graph, per-class lifecycle states, and the event-naming convention. Cross-referenced from this roadmap and from the spec.
- `autolisp-spec` chapter 21 gains the **Host Object Ontology and Lifecycles** normative section: object-class table, lifecycle state machines per class, event-scope rule (per-document vs per-application), veto and persistence policies, vendor evidence.
- New runtime module `clautolisp/autolisp-runtime/source/ontology.lisp`. Defines the `reactor` struct, per-type metadata (`*reactor-type-table*`), registry helpers (`add-reactor-to-document`, `all-document-reactors`, ...), and the public emission API:
  - `signal-document-event`
  - `signal-application-event`
  - `set-document-state` — convenience helper that fires the documented activate/deactivate/close event pairs.
  - `set-application-state` — fires beginQuit / endLogFile.
  - `call-with-event-trace` — test-only helper that captures every emission into a list.
- `runtime-session` gains slots:
  - `application-state` (default `:running`)
  - `application-reactors` (per-application registry)
  - `event-trace` (mutable cell installed by call-with-event-trace)

- `document-namespace` gains slots:
  - `state` (default `:loaded`)
  - `reactors` (per-document registry)
  - `persistent-reactor-index`
  - `session` (back-pointer the event channel uses to resolve a document to its session)
- MockHost mutation paths now emit canonical events:
  - entmake → `:vlr-objectAppended` + `:vlr-objectReappended`
  - entmod → `:vlr-objectModified` + `:vlr-modified`
  - entdel → `:vlr-objectErased` / `:vlr-objectUnErased`
  - setvar → `:vlr-sysVarWillChange` + `:vlr-sysVarChanged`

Phase 14b — Reactor surface (vlr- family)*

- `clautolisp/autolisp-builtins-core` gains 41 new builtins:
  - 20 type-specific constructors — ~vlr-{acdb,command,deepclone,document,dwg,dxf,insert, mouse,object,sysvar,toolbar,undo,wblock,window,xref, docmanager,editor,linker,lisp,miscellaneous}-reactor~
  - Mutation: `vlr-add`, `vlr-remove`, `vlr-remove-all`, `vlr-data`, `vlr-data-set`, `vlr-owners`, `vlr-owner-add`, `vlr-owner-remove`, `vlr-set-notification`, `vlr-notification`, `vlr-current-reaction-name`, `vlr-reactions`, `vlr-reaction-set`, `vlr-trace-reaction`.
  - Introspection: `vlr-added-p`, `vlr-type`, `vlr-types`, `vlr-reactors`.
  - Persistence: `vlr-pers`, `vlr-pers-release`, `vlr-pers-list`, `vlr-pers-p`, `vlr-pers-dictname`.
- Persistent reactors require their callbacks to be `autolisp-symbol` designators (closures cannot survive serialisation); the persistence helper records the failure in the persistent-reactor index keyed on reactor-id.
- Re-entry depth cap (`*reactor-recursion-cap*` = 16) prevents runaway callback chains from saturating the stack.
- Tests:
  - 10 new tests in `autolisp-runtime/tests/ontology-tests.lisp` pin the reactor struct, registry helpers, dispatch by KIND and by reaction-name, the inactive-reactor guard, the lifecycle-state event pairs, the trace-only-when-bound semantics, and the application-state quit events.
  - 4 new tests in `autolisp-mock-host/tests/reactor-tests.lisp` pin end-to-end reactor dispatch through MockHost mutations (entmake/entmod/setvar fires the right canonical events; a registered reactor receives the matching reaction in its callback).
  - Aggregate suite 76 + 206 + 29 + 168 + 428 + 130 = 1037 / 0.

Smoke (under the default `--host mock`):

```
$ clautolisp -x "(princ (vlr-types))"
(ACDB COMMAND DEEPCLONE DOCUMENT DWG DXF INSERT MOUSE OBJECT
 SYSVAR TOOLBAR UNDO WBLOCK WINDOW XREF DOCMANAGER EDITOR LINKER
 LISP MISCELLANEOUS)
```

Phase 15a — DCL Parser, Runtime, and Terminal UI [DONE 2026-04-26]

Goal: dialog-control-language support with a usable fallback renderer that does not require a GUI toolkit.

Realised scope:

- New `clautolisp/autolisp-dcl` system with `package.lisp`, `model.lisp`, `parser.lisp`, `runtime.lisp`, `terminal.lisp`.
- DCL syntax parser: line / block comments, attribute declarations, named children, anonymous children, predefined tile instantiation, string and numeric literals, identifier case-folding via `dcl-keyword`.
- `dcl-tile` / `dcl-source` / `dcl-dialog` records — keyed state map, action map, mode map, client-data map, focus pointer, status, finished-p flag.
- Predefined tile registry seeded with `ok_button`, `cancel_button`, `help_button`, `ok_only`, `ok_cancel`, `ok_cancel_help` (extensible at load-time via `register-predefined-tile`).
- Runtime API: `dcl-runtime-load-dialog`, `dcl-runtime-unload-dialog`, `dcl-runtime-new-dialog`, `dcl-runtime-start-dialog`, `dcl-runtime-done-dialog`, `dcl-runtime-action-tile`, `dcl-runtime-set-tile`, `dcl-runtime-get-tile`, `dcl-runtime-mode-tile`, `dcl-runtime-client-data` / `-set-client-data`, `dcl-runtime-find-tile`, `dcl-runtime-fire-action`.
- Pluggable renderer protocol via `*dcl-renderer*` / `install-default-renderer`. Renderer struct exposes `open-fn`, `close-fn`, `set-tile-fn`, `focus-fn`, `mode-fn`, `run-fn` slots.
- Pure-Common-Lisp terminal renderer (no ncurses, no FFI): ASCII tile-tree rendering, line-oriented prompt loop, `key=value` commands fire actions, `accept~/~cancel~/~quit` exit with conventional status (1/0/0). Auto-installed at load time.
- Action callback contract: `action_tile` accepts either an AutoLISP source string (parsed and evaluated via `read-runtime-from-string` + `autolisp-eval-progn`) or a callable. Before invoking it, `$KEY$`, `$VALUE$`, `$REASON$`, `$DATA$` are bound in the active evaluation context.
- AutoLISP-visible builtins (21): `load_dialog`, `unload_dialog`, `new_dialog`, `start_dialog`, `done_dialog`, `action_tile`, `set_tile`, `get_tile`, `mode_tile`, `client_data_tile`, `dimx_tile`, `dimy_tile`, `start_image`, `end_image`, `fill_image`, `vector_image`, `slide_image`, `start_list`, `add_list`, `end_list`, `term_dialog`.
- Tests: 14 FiveAM tests (parser corpus + runtime lifecycle + action callback symbol bindings + set/get round-trip + default-renderer presence). Total 25 checks, all passing.

Out of scope (Phase 15b):

- `start_list` / `add_list` / `end_list` are placeholders that do not yet feed the list-box renderer.
- Image tiles render as a stub.

- No GUI subprocess driver yet.

Phase 15b — Pluggable GUI Protocol + Qt6 C++ Driver [DONE 2026-04-26]

Goal: a wire protocol that lets a separate process act as the DCL renderer, while keeping the terminal renderer as the documented fallback.

Realised scope:

- Sexp-based wire protocol (`autolisp-dcl/source/sexp-wire.lisp`). Hand-rolled reader and writer — not full CL — so a non-CL driver implements it cheaply. Token grammar: parens, quoted strings with the standard escapes, `:keywords`, integers and floats, `nil` / `t` literals, bare identifiers folded to uppercase keywords. Each message is one s-expression terminated by a newline; readability is a design goal.
- Downstream messages: `(:open-dialog DID NAME TILE-FORM)`, `(:close-dialog DID STATUS)`, `(:set-tile DID KEY VALUE)`, `(:focus DID KEY)`, `(:mode-tile DID KEY MODE)`, `(:bye)`, plus `(:start-list ...)`, `(:add-list ...)`, `(:end-list ...)` reserved.
- Upstream messages: `(:action DID KEY VALUE REASON)`, `(:done DID STATUS)`, `(:hello PROTOCOL-VERSION)`, `(:error MESSAGE)`.
- TILE-FORM grammar: `(:tile TYPE KEY (:attr (NAME VALUE) ...) (:children TILE-FORM ...))`.
- `make-subprocess-renderer`: a `dcl-renderer` implementation that funnels every event over the wire protocol to a child process spawned via `uiop:launch-program`. The subprocess is reaped when the last active dialog closes.
- `--gui CMD` CLI flag and `CLAUTOLISP_GUI` environment variable in `clautolisp-tool`. When neither is set, the built-in terminal renderer remains in effect.
- Reference Qt6 C++ subprocess in `tools/clautolisp-gui-qt` (CMake build): `main.cpp` event loop, `sexp.{hpp,cpp}` reader / writer (mirrors the CL grammar), `wire.{hpp,cpp}` TILE-FORM decoder, `dialog_window.{hpp,cpp}` widget tree builder. Supports `text`, `button`, `edit_box`, `toggle`, `list_box`, `popup_list`, `row`, `column`, `boxed_row`, `boxed_column`, `ok_only`, `ok_cancel`, `ok_cancel_help`.
- 8 new wire-protocol tests (round-trip primitives, escapes, lists, nested forms, multi-message stream, error path, tile encoding) and 3 subprocess-renderer tests (write-end formatting + tile-encoding shape + write/read symmetry). Total 57 DCL checks, all passing.

Out of scope (next phases):

- `start_image` / `vector_image` / `slide_image` widgets are not yet rendered by the Qt6 driver; messages are reserved but not produced.
- `start_list` / `add_list` / `end_list` on the runtime side remain placeholder no-ops; the wire-protocol reservation is ready for when a real DCL

application exercises them.
- No autobuild of the Qt6 driver in the project Makefile — CMake usage is documented in `tools/clautolisp-gui-qt/README.org`.

Phase 16 — LiveHost Adapter

Goal: run AutoLISP scripts through clautolisp against an actual BricsCAD or AutoCAD process.

Scope (highly speculative):

- Stdio-bridge prototype against BricsCAD (Linux / macOS are feasible without ActiveX).
- COM bridge for AutoCAD on Windows.
- Integration with the existing `autolisp-spec` probe harness.

Cross-Cutting Concerns

Coverage tracking

A new tooling target `make builtins-coverage` produces a report comparing `core-builtins` entries against the function inventory in `autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org`, broken down by chapter. Each phase above closes a column of that report.

Testing strategy

- **Unit tests** (FiveAM) per builtin family.
- **Snapshot tests** against MockHost: a script runs, the post-condition entity table is dumped, and the dump is compared against a stored `.snapshot` file.
- **Probe replay** against the BricsCAD V26 / macOS harness for semantics that are not directly testable headlessly (used at spec-time, not at every build).
- **File-compat scenarios** (`autolisp-file-compat`) continue to function as a declarative scenario corpus.

Compatibility profile vs dialect

The dialect descriptor (`strict`, `autocad-2026`, `bricscad-v26`) describes the **language**. The HAL backend describes the **host**. They are independent: a script under the `autocad-2026` dialect can run against MockHost; a script under `strict` can run against LiveHost. Tests must exercise the cross product.

Versioning

`clautolisp/tools/clautolisp/source/version.lisp` bumps DEVELOP on every source change. Phase boundaries optionally bump MINOR (0.0.x → 0.1.0 → ...). MAJOR is reserved for API-breaking changes (e.g., a HAL contract revision).

Open Architectural Questions

These are recorded here so they are remembered when their phase arrives:

1. **ENAME identity across sessions.** If LiveHost reconnects, are ENAMEs from a previous session re-resolvable? Probably no: every session gets a fresh ID space.
2. **Concurrent sessions.** Today's runtime is single-session. Is a pool of MockHosts useful for parallel test execution? Likely yes — `runtime-session` already isolates sessions; the open work is making MockHost itself thread-safe.
3. **Garbage collection of pickset / VLA tables.** A long-running session leaks. Provide an explicit `clear-host-handles` helper on MockHost; let LiveHost translate it to `object-released-p` sweeps.
4. **`(command ...)` interactivity in headless mode.** The `command` builtin in real AutoCAD drives the command line. In MockHost we record commands to a log; `initget` / `getstring`-like prompts feeding into `command` need a coherent contract.
5. **DCL rendering.** Headless DCL records actions. A production GUI backend will need to choose between Qt, web, or platform-native; the HAL contract should be neutral on that question.
6. **COM type library scope.** A real AutoLISP corpus references a long tail of `vla-*` functions generated from imported type libraries. We will have to draw a line between "documented" and "user-imported" and only ship the documented surface in core.

Cross-References

- **Spec**: `autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org` — chapter index drives every phase.
- **Existing PLAN**: `clautolisp/PLAN.md` — actionable tasks. This roadmap is the parent document.
- **Sibling design notes**: `clautolisp/documentation/development-plan.org` — earlier high-level architecture rationale.
- **User manual**: `clautolisp/documentation/user-manual.org` — the CLI surface, kept in sync with `tools/clautolisp/source/main.lisp`.